VERIQTA

INFRASTRUCTURE ENGINEER'S DAILY OPERATIONS HANDBOOK

YOUR PRACTICAL GUIDE TO KEEPING SYSTEMS HEALTHY, SECURE, AVAILABLE, AND RELIABLE EVERY DAY

SERVERS | STORAGE | NETWORKING | SECURITY | MONITORING

DAILY CHECKS & ROUTINES | TROUBLESHOOT FASTER | AUTOMATE SMARTER | OPERATE SECURELY | DOCUMENT & IMPROVE

REAL-WORLD TASKS.
PROVEN PRACTICES.
BETTER OPERATIONS.

From junior engineer to trusted operator.

OBSERVE. INVESTIGATE. CHANGE. VERIFY.
DOCUMENT. MONITOR. HANDOVER.

@veriqta | veriqta | veriqta | veriqta | @veriqta | @veriqta

THE INFRASTRUCTURE ENGINEER'S DAILY OPERATIONS

1 WHAT INFRASTRUCTURE ENGINEERS ACTUALLY MANAGE

SERVERS | COMPUTE | STORAGE | NETWORKING | IDENTITY & ACCESS | VIRTUALIZATION | CLOUD PLATFORMS

We keep systems healthy, secure, available and performant so applications and users can work without disruption.

2 OPERATIONS VS PROJECTS

OPERATIONS	PROJECTS
Keep systems running	Build or improve systems
Monitor & respond	Plan & design
Troubleshoot issues	Implement new solutions
Perform maintenance	Larger changes
Handle incidents	Have a clear start and end
Small, frequent changes	Focus on delivery
Focus on stability	
GOAL: KEEP THE BUSINESS RUNNING	GOAL: BUILD THE FUTURE STATE

3 TYPICAL DAILY RESPONSIBILITIES

MONITOR HEALTH & ALERTS
Check dashboards, metrics, and system status.

RESPOND TO INCIDENTS
Investigate alerts and user-reported issues.

PERFORM CHANGES
Apply updates, configuration or fixes safely.

ENSURE SECURITY & COMPLIANCE
Review access, patches, backups and logs.

DOCUMENT EVERYTHING
Record actions, findings and outcomes.

COLLABORATE
Work with DevOps, Network, Security, Developers and other teams.

4 PRODUCTION OWNERSHIP

In production, your decisions have real impact. You are trusted to protect uptime, data, security and performance. Think before you act, verify every change, and always have a rollback plan.

REMEMBER
People may not see your work, but they feel it when systems are slow, down, or insecure.

5 THE DAILY OPERATIONS CYCLE

MORNING
Review dashboards, alerts, backups, tickets and handover notes.

→ **CHANGES**
Implement approved changes, updates or maintenance tasks.

→ **INCIDENTS**
Investigate issues, find root cause and restore service.

→ **VERIFICATION**
Test, validate and confirm everything is working as expected.

→ **HANDOVER**
Document actions and update the next engineer clearly.

JUNIOR ENGINEER TIP

NEVER MAKE A CHANGE YOU CANNOT VERIFY.
VERIFY BEFORE, DURING AND AFTER EVERY CHANGE.

START OF SHIFT, THE INFRASTRUCTURE HEALTH CHECK

Your morning check sets the tone for the entire day.
Find problems early, prevent incidents, and keep systems reliable.

1 REVIEW MONITORING DASHBOARDS

Check dashboards for overall system health, metrics trends and warning signs.

2 OVERNIGHT ALERTS AND INCIDENTS

Review all alerts and incidents from the previous shift and confirm their current status.

3 FAILED JOBS

Check CI/CD pipelines, scheduled tasks, cron jobs and automation runs that may have failed.

4 BACKUP STATUS

Verify backups completed successfully and are within their retention and schedule windows.

5 RESOURCE HEALTH

Check CPU, memory, disk usage and network health across critical servers and systems.

6 SERVICE AVAILABILITY

Confirm key services, applications and endpoints are up, responsive and performing normally.

7 SCHEDULED MAINTENANCE

Review planned maintenance windows, updates, reboots and change activities for the day.

8 OPEN TICKETS

Review open tickets, prioritize issues and check for updates or new comments.

TICKET

9 PREVIOUS HANDOVERS

Read handover notes carefully. Understand what was done, what is in progress and what needs attention.

BUILD A REPEATABLE MORNING CHECKLIST

Consistency reduces missed problems, improves response time, and builds trust.

JUNIOR ENGINEER TIP
THE EARLIER YOU FIND AN ISSUE, THE CHEAPER AND EASIER IT IS TO FIX.

SERVER HEALTH AND RESOURCE CHECKS

Healthy servers are the foundation of everything. Check these resources every day.

CPU Utilization and Load | Memory and Swap | Disk Utilization | Filesystem Health | Uptime | Running Processes

ESSENTIAL COMMANDS YOU USE EVERY DAY

`uptime` `$ uptime`	Shows how long the system has been running, current time, logged in users and load average.
`free -h` `$ free -h`	Displays total, used and free memory and swap in human readable format.
`df -h` `$ df -h`	Shows disk space usage for all mounted filesystems.
`top` `$ top`	Real-time view of running processes and system resource usage.
`ps` `$ ps aux`	Shows currently running processes. Use with options to filter and sort.

QUICK INTERPRETATION GUIDE

CPU > 80% for long periods	=	Investigate high CPU process
Memory > 80% or Swap in use	=	Add memory or analyze usage
Disk > 85%	=	Clean up or extend storage
Inodes > 90%	=	Too many files, clean up
Load Avg > CPU cores	=	System under heavy load
Uptime very low	=	Frequent reboots, check stability

RECOGNIZING ABNORMAL RESOURCE CONSUMPTION

HIGH CPU
- CPU consistently above 80–90%
- System feels slow or unresponsive

HIGH MEMORY USAGE
- Used memory close to total
- Processes start using swap

HIGH SWAP USAGE
- Any constant swap usage is bad
- Causes slow performance

HIGH DISK USAGE
- Disk usage above 85–90%
- Applications may fail or crash

FILESYSTEM ISSUES
- Read-only filesystem (ro)
- Inode usage close to 100%

HIGH LOAD AVERAGE
- Load average much higher than CPU cores
- Indicates system stress

SUSPICIOUS PROCESSES
- Unknown or unexpected processes
- Processes using too much CPU, memory or disk

JUNIOR ENGINEER TIP

NEVER MAKE A CHANGE YOU CANNOT VERIFY. CHECK, UNDERSTAND, THEN ACT.

PROCESSES AND SERVICES IN DAILY OPERATIONS

PROCESS(ES) are running programs.
SERVICE(S) are long-running processes managed by systemd.

1 PROCESS VS SERVICE

PROCESS	SERVICE
• Started manually • Runs in foreground • Stops when terminal closes • Not automatically restarted	• Managed by systemd • Runs in background • Starts on boot • Monitored and automatically restarted if it fails

2 SYSTEMD – THE SERVICE MANAGER

systemd is PID 1 on most modern Linux systems. It manages services, starts on boot, monitors health and handles dependencies.

COMMON SERVICE STATES

active (running)	Service is running normally
inactive (dead)	Service is not running
failed	Service failed to start or crashed
activating	Service is starting
deactivating	Service is stopping

3 ESSENTIAL SYSTEMD COMMANDS

`systemctl status <service>`	Check the status and health of a service. Shows state, PID, uptime and recent logs.
`systemctl restart <service>`	Restart a service safely. Use with caution in production.
`systemctl list-units --failed`	List all services that are in failed state.
`journalctl -u <service>`	View logs for a specific service. Use -f to follow logs live.
`journalctl -xe`	View system logs with errors and boot issues.
`systemctl enable <service>`	Enable a service to start automatically on boot.
`systemctl disable <service>`	Disable a service from starting on boot.

4 UNDERSTANDING FAILED SERVICES

- Check status: `systemctl status <service>`
- Read logs: `journalctl -u <service> --no-pager`
- Look for errors, misconfigurations, missing files, permissions, port conflicts, or dependency failures.
- Do not guess – collect evidence.

RESTARTING SAFELY

- Verify impact before restarting critical services.
- Check if the service is already failing.
- Prefer graceful reloads (if supported).
- Communicate in team or ticket.
- Monitor after restart to confirm stability.

RESTARTING IS NOT TROUBLESHOOTING

- Restarting may hide the real issue.
- The service could fail again.
- Always find the root cause instead of using restart as a quick fix.
- Fix the problem, then restart if needed.

JUNIOR ENGINEER TIP

RESTARTING FIXES SYMPTOMS.
TROUBLESHOOTING FIXES THE CAUSE.

- CHECK STATUS
- CHECK LOGS
- UNDERSTAND ISSUE
- TAKE CORRECT ACTION
- VERIFY RESULT

LOGS, YOUR FIRST SOURCE OF EVIDENCE

Logs tell you WHAT happened, WHEN, and WHY.
They are the truth. Always check the logs first.

1 WHY INFRASTRUCTURE GENERATES LOGS

Logs record events, errors, changes and activities so engineers can monitor systems, troubleshoot issues, ensure security and meet compliance requirements.

2 TYPES OF LOGS

SYSTEM LOGS
Events from the OS and system services (e.g., systemd, cron, packages, boot).

APPLICATION LOGS
Logs from installed applications and services (web servers, databases, APIs).

AUTHENTICATION LOGS
Login attempts, sudo usage, SSH access, access denials and security events.

KERNEL LOGS
Low-level events from the Linux kernel (drivers, hardware, errors, warnings).

3 ESSENTIAL LOG VIEWING COMMANDS

`journalctl`	View and filter systemd and system logs. `journalctl -xe`
`tail`	View the last part of a log file. `tail -n 100 /var/log/syslog`
`less`	Open and navigate through log files. `less /var/log/syslog`
`grep`	Search for patterns or errors inside logs. `grep -i error /var/log/syslog`
`tail -f`	Follow logs in real time (live streaming). `tail -f /var/log/nginx/access.log`

4 TIMESTAMPS AND CORRELATION

- Every log entry has a timestamp.
- Use timestamps to build a timeline.
- Correlate events across system, application and network logs.
- Time sync (NTP) must be correct across all servers.

5 SEARCHING THE RIGHT WAY

- Do not guess — search.
- Look for ERROR, FAIL, DENIED, TIMEOUT, EXCEPTION, OOM, CRASH and WARN.
- Use specific keywords and time ranges.
- Read surrounding lines for context.

6 COMMON LOG LOCATIONS

- /var/log/syslog or /var/log/messages
- /var/log/auth.log
- /var/log/kern.log
- /var/log/dmesg
- /var/log/secure
- /var/log/<application>/

FIND THE ACTUAL FAILURE, NOT JUST THE SYMPTOM.
LOGS DON'T LIE. GUESSING DOES.

JUNIOR ENGINEER TIP
ALWAYS CHECK LOGS FIRST.
IT SAVES TIME, PREVENTS WRONG ACTIONS AND SOLVES REAL PROBLEMS.

DISK, FILESYSTEM AND STORAGE OPERATIONS

Storage problems can stop your systems.
Understand usage, capacity and where data lives.

1 KEY CONCEPTS

DISK CAPACITY VS FILESYSTEM USAGE

Disk is the physical drive. Filesystem is what is mounted and used.

MOUNT POINTS

Directories where filesystems are attached to the directory tree.

Example: /, /var, /home, /data

INODES

Inodes store file metadata. You can run out of inodes even when space is available.

FILESYSTEM HEALTH

A full filesystem or out-of-inodes can crash apps and services.

MONITOR REGULARLY

Check usage daily. Catch problems before users feel them.

2 ESSENTIAL COMMANDS

`df -h`	Show disk space usage in human readable format. Check capacity and usage for all mounted filesystems.
`du -sh <path>`	Show total size of a directory or file in human readable format. Useful for finding big folders.
`lsblk`	List block devices and their mount points. Understand disk layout.
`mount`	Show mounted filesystems and details. Verify what is mounted and where.
`df -i`	Show inode usage. Important when you see "No space left on device" but `df -h` looks fine.

3 FINDING LARGE FILES & DIRECTORIES

- Find large directories (top level only)

```
du -h --max-depth=1 / | sort -hr
```

- Find large files

```
find / -type f -size +500M -exec ls -lh {} \;
```

- Find and sort by size

```
du -ahx / | sort -rh | head -n 20
```

4 DISK-FULL INCIDENTS

- Applications fail unexpectedly.
- Services cannot write logs or data.
- Users cannot create or save files.
- Databases and queues may crash.
- Root filesystem full = system unavailable.

ACT FAST, BUT ACT CORRECTLY.

5 SAFE CLEANUP PRACTICES

- ✓ Identify what is using space before deleting.
- ✓ Clean package caches safely.

```
sudo apt-get clean || yum clean all
```

- ✓ Rotate or compress logs.
- ✓ Remove old backups that are outside retention.
- ✓ Use logrotate for automatic management.
- ✓ Ask before deleting shared or critical data.

CLEAN SMART, NOT FAST.

6 WHY BLINDLY DELETING LOGS IS DANGEROUS

- ✗ You may delete evidence needed for incidents.
- ✗ You might remove logs applications depend on.
- ✗ 3rd party tools may rely on specific log files.
- ✗ You can break compliance and retention policies.
- ✗ You may delete the wrong logs and hide the real issue.

ALWAYS UNDERSTAND BEFORE YOU DELETE.

JUNIOR ENGINEER TIP

CHECK → UNDERSTAND → CLEAN → VERIFY → MONITOR

NEVER DELETE ANYTHING YOU DO NOT UNDERSTAND.

- ✓ CHECK USAGE
- ✓ IDENTIFY CAUSE
- ✓ CLEAN SAFELY
- ✓ VERIFY SPACE
- ✓ MONITOR AFTER

NETWORKING CHECKS
EVERY INFRASTRUCTURE ENGINEER USES

IP ADDRESSES | INTERFACES | ROUTES | DNS | PORTS | CONNECTIONS

ESSENTIAL NETWORKING COMMANDS

COMMAND	PURPOSE	EXAMPLE	WHAT IT SHOWS
`ip addr`	Show IP addresses and interfaces	`ip addr`	IP addresses, interface status, MAC address, MTU
`ip route`	Show routing table	`ip route`	Default gateway, routes to networks and destinations
`ping`	Test basic network connectivity	`ping 8.8.8.8`	Reachability, packet loss, round-trip time
`ss`	List open ports and connections	`ss -tulnp`	Listening ports, established connections, PIDs
`curl`	Test HTTP/HTTPS connectivity	`curl -I https://example.com`	HTTP response, status codes, headers, connectivity
`dig`	Query DNS records	`dig example.com`	DNS resolution, record types, IP addresses, TTL
`traceroute`	Trace the path to a destination	`traceroute example.com`	Every hop, latency, where packets stop

DISTINGUISHING COMMON NETWORKING FAILURES

DNS FAILURE

dig domain.com fails or returns NXDOMAIN.

Symptoms:
- Website name not found
- Applications cannot resolve domains

Check:
- DNS server
- dig / nslookup

ROUTING FAILURE

Ping IP fails beyond certain network.

Symptoms:
- Cannot reach remote IP
- Works within network only

Check:
- ip route
- traceroute

FIREWALL / PORT BLOCKED

Port unreachable or connection refused.

Symptoms:
- Connection timeout
- Port unreachable
- Connection refused

Check:
- Firewall rules
- ss -tulnp on target

APPLICATION FAILURE

Network is fine, but app is not responding correctly.

Symptoms:
- HTTP 5xx / 4xx errors
- App not loading
- Service unhealthy

Check:
- curl -I http://ip:port
- Application logs

JUNIOR ENGINEER TIP

CHECK IN ORDER: DNS → ROUTING → PORT → APPLICATION.
DO NOT GUESS. TEST EACH LAYER AND FIND THE REAL PROBLEM.

SSH AND REMOTE SERVER ADMINISTRATION

1 HOW SSH ACCESS WORKS

Your client authenticates to the server, then all traffic is encrypted.

2 KEY CONCEPTS

SSH KEYS	Authenticate without passwords using public/private key pairs.
ssh	Open a secure remote shell session.
scp	Securely copy files between systems.
~/.ssh/config	Simplify and organize SSH connections.
KNOWN HOSTS	Ensures you connect to the correct and trusted server.

3 ESSENTIAL SSH COMMANDS

COMMAND	PURPOSE
`ssh user@server-ip`	Connect to a remote server.
`ssh -i key.pem user@server-ip`	Connect using a specific private key.
`scp file.txt user@server:/path/`	Copy a file to remote server.
`scp user@server:/path/file.txt .`	Copy a file from remote server.
`ssh -L 8080:localhost:80 user@server`	Local port forwarding.
`ssh -N -f -L 3306:db:3306 user@server`	Background tunnel.

4 ~/.SSH/CONFIG EXAMPLE

```
Host prod-web
   HostName 203.0.113.10
   User ubuntu
   IdentityFile ~/.ssh/id_ed25519
   Port 22

Host prod-db
   HostName 203.0.113.20
   User ubuntu
   IdentityFile ~/.ssh/id_ed25519
   Port 22
```

5 PERMISSION REQUIREMENTS

Private key: `chmod 600 ~/.ssh/id_ed25519`

.ssh directory: `chmod 700 ~/.ssh`

6 COMMON SSH ERRORS

PERMISSION DENIED (PUBLICKEY)

Server cannot find or accept your public key.

Check:
- Key pair is correct
- Public key is in ~/.ssh/authorized_keys
- Correct permissions
- Correct username

CONNECTION TIMEOUT

No response from server.

Check:
- Network connectivity
- Firewall rules
- Server is running
- Correct IP and port

CONNECTION REFUSED

Server is reachable but SSH is not accepting connections.

Check:
- SSH service status
- Firewall / Security Group
- Correct port (usually 22)

7 PROTECTING PRIVATE KEYS

- Never share your private key.
- Do not commit keys to Git or anywhere.
- Use strong passphrases.
- Store keys securely and limit access.
- Rotate keys if compromised.
- Use least-privilege accounts.

WHY PRODUCTION SSH ACCESS SHOULD BE CONTROLLED

- Prevent unauthorized access.
- Reduce risk of data breaches.
- Ensure accountability and auditability.
- Limit blast radius of compromised accounts.
- Enforce security policies and compliance.

9 BEST PRACTICES

- Use SSH keys, not passwords.
- Use dedicated non-root users.
- Enable key-based auth and disable password auth.
- Log and monitor SSH access.
- Review access regularly.

JUNIOR ENGINEER TIP

ALWAYS VERIFY BEFORE AND AFTER EVERY CHANGE.
SECURE ACCESS TODAY PREVENTS INCIDENTS TOMORROW.

USERS, GROUPS, PERMISSIONS AND PRIVILEGED ACCESS

USERS	GROUPS	OWNERSHIP	PERMISSIONS	ACCESS	PRIVILEGED ACCESS
Individual accounts that access the system.	Collections of users to simplify permission management.	Every file and directory has an owner and a group owner.	Define what users (owner, group, others) can do.	Control who can do what, and how much.	Use elevated rights only when necessary and for the right amount of time.

FILE PERMISSIONS EXPLAINED

Linux permissions use three categories:

Owner (u) Group (g) Others (o)

And three types of access:

Read	(r)	=	4
Write	(w)	=	2
Execute	(x)	=	1

EXAMPLE: -RWXR-XR--

-	r w x	r - x	r - -
	Owner	Group	Others
Numeric:		7 5	4

COMMON NUMERIC PERMISSIONS

rwxr-xr-x	755
rwxrwxr-x	775
rw-r--r--	644
rw-------	600

ESSENTIAL COMMANDS

COMMAND	PURPOSE	EXAMPLE	WHAT IT SHOWS / DOES
id	Show current user and group information	`id`	UID, GID, groups you belong to
ls -l	List files with detailed permissions	`ls -l /var/log`	Permissions, owner, group, size, date, filename
chown	Change file or directory ownership	`sudo chown user:group file.txt`	Changes owner and group of a file or directory
chmod	Change file or directory permissions	`sudo chmod 755 script.sh`	Changes read/write/execute permissions
sudo	Run a command as another user (usually root)	`sudo systemctl restart nginx`	Execute privileged commands securely

LEAST PRIVILEGE PRINCIPLE

- Give users only the access they need.
- Remove unnecessary permissions.
- Use groups to manage access.
- Review and revoke access regularly.

ACCESS REVIEWS

- Review user accounts.
- Review group memberships.
- Verify sudo access.
- Remove stale or unused access.
- Document and audit changes.

WHY CHMOD 777 IS NOT OK

- Opens full access to everyone.
- Creates security risks.
- Can be exploited by attackers.
- Masks the real problem.
- Makes troubleshooting harder.

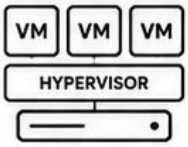
JUNIOR ENGINEER TIP

USE THE LEAST PRIVILEGE ACCESS THAT WORKS.
AVOID CHMOD 777. FIX THE ROOT CAUSE, NOT THE SYMPTOM.

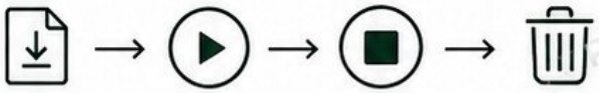
VIRTUAL MACHINES, CLOUD INSTANCES AND COMPUTE

1 PHYSICAL VS VIRTUAL INFRASTRUCTURE

PHYSICAL (BARE METAL)	VIRTUAL (VMs)
• Dedicated hardware • You manage OS, apps and everything • Higher cost to scale • Longer provisioning time	• Multiple VMs on one host • Isolation with efficiency • Faster provisioning • Easier to scale and manage



2 VM LIFECYCLE



CREATE — Provision and configure.

RUNNING — VM is active and in use.

STOPPED — VM is off but resources are preserved.

TERMINATED — VM is deleted and resources are released.

A stopped VM can be started again.
A terminated VM must be recreated.

3 CLOUD COMPUTE INSTANCES

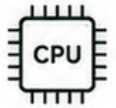
aws · Microsoft Azure · Google Cloud

- On-demand virtual servers in the cloud.
- Pay-as-you-go pricing model.
- Highly available, scalable, and resilient.

4 INSTANCE STATES

RUNNING	Instance is active and incurring cost.
STOPPED	Instance is stopped. You pay for storage.
TERMINATED	Instance is deleted. All resources released.

5 CPU AND MEMORY SIZING



MEMORY

- Right-size for workload requirements.
- Over-provisioning wastes money.
- Under-provisioning causes poor performance.
- Monitor usage and adjust as needed.

6 IMAGES (TEMPLATES)

- Pre-configured OS and software.
- Use official images or hardened custom images.
- Keep images updated and secure.

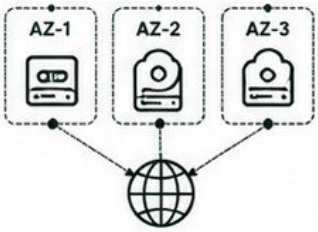
7 INSTANCE METADATA

- Data about the instance provided by the cloud.
- Includes IP, hostname, tags, IAM role, user-data.
- Used by apps and scripts running on the instance.

8 SECURITY GROUPS / FIREWALLS

- Control inbound and outbound traffic.
- Allow only required ports and protocols.
- Always follow least privilege.

9 AVAILABILITY ZONES (AZs)



- AZs are isolated locations within a region.
- Spread instances across AZs for high availability.
- Protects against data center failures.

10 CHECKING UNHEALTHY INSTANCES

- Check system metrics (CPU, memory, disk, network).
- Review logs and application health.
- Verify connectivity and service status.
- Identify root cause before taking action.

11 AVOID UNNECESSARY RESTARTS

- Restarting hides the real problem.
- Investigate before you restart.
- Restarts can cause downtime and data loss.
- Fix the cause, not the symptom.

12 DAILY CHECKLIST FOR COMPUTE HEALTH

Check instance health and metrics

Review security groups/firewalls

Verify disk and storage

Check backups and snapshots

Review tags, metadata and IAM

Confirm monitoring and alerts

Validate service availability

JUNIOR ENGINEER TIP
ALWAYS UNDERSTAND THE IMPACT BEFORE YOU START, STOP OR TERMINATE ANY INSTANCE. INVESTIGATE FIRST, RESTART LAST.

DNS, LOAD BALANCERS AND APPLICATION REACHABILITY

1 REQUEST PATH MENTAL MODEL

CLIENT → DNS → LOAD BALANCER → SERVER / APPLICATION

DNS
Resolves domain to IP address

LOAD BALANCER
Distributes traffic to healthy backends

SERVER / APPLICATION
Processes request and returns response

MENTAL MODEL

Break the path into layers. Test each layer in order to find where the failure is actually happening.

2 DNS RECORDS

- A – Maps domain to IPv4 address
- AAAA – Maps domain to IPv6 address
- CNAME – Points one name to another
- MX – Mail exchange records
- TXT – Text records (SPF, DKIM, etc.)
- NS – Name server records

3 HEALTH CHECKS

- Load balancer checks backend health regularly.
- Only healthy backends receive traffic.
- Unhealthy backends are removed automatically.

4 BACKEND TARGETS

- Backend targets are the actual servers/instances.
- Targets can be IPs or instance IDs.
- Incorrect targets = 502 / 503 errors.

5 PORTS

Common ports:

- 80 HTTP
- 443 HTTPS
- 22 SSH
- 3306 MySQL
- 5432 PostgreSQL
- 6379 Redis

Verify the correct port is open and listening on the backend.

6 TLS BASICS

TLS

- TLS encrypts traffic (HTTPS = HTTP over TLS).
- Certificate validates identity.
- Expired or invalid certs break trust.
- Handshake failures = connection errors.

7 TESTING WITH dig AND curl

dig

```
dig example.com
```

- Check DNS resolution
- Verify record type
- Validate DNS servers

curl

```
curl -I https://example.com
```

- Test HTTP/HTTPS response
- Check status codes
- See headers and redirects

8 DIAGNOSING "THE WEBSITE IS DOWN"

DNS FAILURE

Domain does not resolve or resolves to wrong IP.

Check:
- dig domain.com
- DNS record
- Propagation

ROUTING FAILURE

Network cannot reach the destination.

Check:
- ping IP
- traceroute
- Route tables

FIREWALL / PORT BLOCKED

Port is blocked somewhere.

Check:
- Security groups
- Firewall rules
- Port accessibility

LOAD BALANCER HEALTH FAILURE

No healthy backends or misconfigured health checks.

Check:
- LB health checks
- Backend status
- Target group

APPLICATION FAILURE

Application is down or returning errors.

Check:
- Server logs
- Application logs
- Status codes

STATUS CODE QUICK GUIDE

200	OK – Success
301 / 302	Redirect
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error
502	Bad Gateway
503	Service Unavailable

9 FINDING WHICH LAYER ACTUALLY FAILED

START WITH THE CLIENT ERROR
What exactly are users seeing?

→ **TEST DNS**
dig domain.com
Does it resolve?

→ **TEST CONNECTIVITY**
ping IP
traceroute IP

→ **TEST PORT**
nc -vz IP PORT
or curl -I

→ **TEST APPLICATION**
Check status code, response and logs.

→ **CONFIRM AND DOCUMENT**
Record findings and share resolution.

JUNIOR ENGINEER TIP

ALWAYS TEST LAYER BY LAYER. GUESSING WASTES TIME.
IDENTIFY THE BROKEN LAYER, FIX THE REAL PROBLEM.

DO NOT RESTART SERVICES UNNECESSARILY.
FIND THE ROOT CAUSE FIRST.

BACKUPS, SNAPSHOTS AND RESTORE VERIFICATION

1 BACKUP vs SNAPSHOT

BACKUP	SNAPSHOT
Copies data to a separate location	Point-in-time image of data on the same storage
Can be stored off-site or in the cloud	Very fast and space efficient
Used for long-term protection	Used for quick recovery or rollback
Survives ransomware, hardware failure, or disaster	Does not protect against site-wide failure or data loss

2 RPO vs RTO

RPO (RECOVERY POINT OBJECTIVE)
How much data you can afford to lose.
Example: RPO = 15 minutes
→ You may lose up to 15 minutes of data.

RTO (RECOVERY TIME OBJECTIVE)
How fast a system must be restored.
Example: RTO = 2 hours
→ The system must be back within 2 hours.

REMEMBER
RPO is about data loss.
RTO is about downtime.

3 KEY BACKUP BEST PRACTICES

BACKUP SCHEDULES
Set regular schedules that match your RPO and business needs.

RETENTION
Keep enough history to recover from human errors or data corruption.

OFF-SITE COPIES
Store backups in a separate location or cloud account for disaster protection.

ENCRYPTION
Encrypt backups in transit and at rest to protect sensitive data.

MONITOR & ALERTS
Monitor backup jobs and get alerts for failures or delays.

ACCESS CONTROL
Restrict who can create, access and delete backups.

4 FAILED BACKUP JOBS

- Disk full or insufficient space
- Network connectivity issues
- Permission or authentication failures
- Backup service or agent down
- Timeouts or performance bottlenecks
- Misconfigured backup paths or policies

**ALWAYS INVESTIGATE AND FIX THE ROOT CAUSE.
A FAILED BACKUP TODAY IS A DISASTER TOMORROW.**

5 RESTORE TESTING

- Regularly test restores in a safe environment.
- Verify data integrity and completeness.
- Test full restore and point-in-time restore.
- Document the restore process.
- Know the time and steps it takes.

TESTING PROVES YOUR BACKUP WORKS BEFORE YOU NEED IT.

6 WHY "BACKUP COMPLETED" DOES NOT PROVE DATA CAN BE RECOVERED

- Backup files can be corrupted or incomplete.
- Restoration process may fail.
- Permissions may prevent access.
- Data may be encrypted but keys are missing.
- Application may not start after restore.
- Configuration or dependencies may be missing.

A BACKUP IS ONLY AS GOOD AS A SUCCESSFUL RESTORE.

VERIFY, TEST, AND TRUST YOUR RECOVERY PROCESS.

MONITORING, METRICS AND ALERT RESPONSE

1 INFRASTRUCTURE OBSERVABILITY

Observability is knowing what is happening in your systems based on data, not guesses.

Collect → Measure → Analyze → Understand → Act

2 KEY INFRASTRUCTURE METRICS

CPU	MEMORY	DISK	NETWORK
Utilization % Load average	Usage % Available	Usage % IOPS, Latency	Bandwidth Errors, Drops

AVAILABILITY METRICS

- Uptime / Downtime
- Service availability
- Error rate (5xx, timeouts)
- Response time / Latency
- SLA / SLO compliance
- Health check success rate

3 DASHBOARDS

- Single source of truth
- Real-time visibility
- High-level overview
- Drill-down capability
- Know your environment at a glance

4 THRESHOLDS AND ALERTS

METRICS — Data is collected from systems.

→ **THRESHOLDS** — Define normal vs abnormal behavior.

→ **ALERTS** — Triggered when thresholds are breached.

→ **NOTIFIED** — The right person gets the alert.

→ **ACTION TAKEN** — Investigate and resolve the issue.

→ **VERIFIED** — Confirm the system is healthy again.

5 WARNING vs CRITICAL

WARNING

- Something needs attention.
- System is still operational.
- Act before it becomes critical.

CRITICAL

- Service or system is at risk.
- Immediate action is required.
- Users may already be impacted.

6 ALERT FATIGUE

Too many noisy or irrelevant alerts cause teams to ignore important ones.

REDUCE ALERT FATIGUE

- Alert on symptoms that matter
- Use sensible thresholds
- Group and suppress similar alerts
- Route alerts to the right people
- Review and tune regularly

7 BASELINES

- Understand normal behavior over time.
- Baseline helps you set smart thresholds.
- Every environment is different.
- Without baselines, you alert on noise.

8 CHECK EVIDENCE BEFORE REACTING

- Don't just react to the alert.
- Check dashboards, logs, metrics and context.
- Verify the impact and scope.
- Then take the right action.

9 THE ALERT RESPONSE FLOW

METRIC — A metric crosses the threshold.

→ **ALERT** — An alert is triggered.

→ **INVESTIGATION** — Check dashboards, logs, metrics and context.

→ **ACTION** — Fix, mitigate or restore the service.

→ **VERIFICATION** — Confirm the system is back to normal.

JUNIOR ENGINEER TIP

Never trust a single alert.
Collect evidence, understand the impact, then act with confidence.

PATCHING AND OPERATING SYSTEM MAINTENANCE

1 WHY SYSTEMS REQUIRE PATCHING

- Fix security vulnerabilities
- Improve stability and performance
- Add new features and enhancements
- Ensure compatibility with other systems and software
- Reduce risk of downtime and data loss
- Maintain compliance and support

NO PATCHING = HIGHER RISK
Unpatched systems are easy targets.

2 TYPES OF UPDATES

SECURITY UPDATES	PACKAGE UPDATES
Fix known vulnerabilities	Bug fixes and improvements
Critical for protection	New features
Apply as soon as possible	Dependency updates
Do not delay security patches	Improve system reliability

3 MAINTENANCE WINDOWS

- Schedule patches during low-traffic periods.
- Communicate with stakeholders.
- Define start time, end time and rollback plan.
- Respect business and uptime requirements.

NO WINDOW = HIGH RISK
Patching during peak hours can cause outages.

4 DEPENDENCY RISKS

- Updates may break applications or services.
- Dependencies may conflict after updates.
- Check release notes and known issues.
- Understand impact before applying.

ALWAYS REVIEW DEPENDENCIES BEFORE PATCHING.

5 TESTING BEFORE PRODUCTION

- Test updates in a staging or pre-production environment.
- Validate functionality, performance and compatibility.
- Reproduce critical workflows.
- Do not patch untested systems.

6 BACKUPS BEFORE RISKY CHANGES

- Take full backups or snapshots.
- Verify backup integrity.
- Ensure backups are restorable.
- Backups are your safety net.

7 REBOOT REQUIREMENTS

- Some patches require a reboot.
- Check if a reboot is needed.
- Plan reboots within the maintenance window.
- Verify system is back online after reboot.

8 POST-PATCH VALIDATION

- Verify services are running.
- Check application health.
- Monitor logs for errors.
- Validate performance and connectivity.
- Confirm business impact is zero.

9 ROLLBACK PLANNING

- Know how to roll back before you patch.
- Have rollback steps documented.
- Keep previous packages or snapshots.
- Test rollback process in non-production.

A GOOD PLAN = FAST RECOVERY

10 AVOID UNCONTROLLED PRODUCTION PATCHING

- No emergency patching without a plan.
- No patching without backups.
- No patching without testing.
- No patching outside maintenance windows.
- No patching without communication.

UNCONTROLLED PATCHING CAUSES OUTAGES.

JUNIOR ENGINEER TIP

PATCH SMART, NOT HARD.

PLAN AHEAD.
TEST FIRST.
BACKUP ALWAYS.
VERIFY EVERYTHING.

THE PATCHING SUCCESS FLOW

PLAN — Define window and scope → **TEST** — Test in staging environment → **BACKUP** — Take and verify backups → **PATCH** — Apply updates safely → **REBOOT** — If required, reboot → **VALIDATE** — Verify system health → **MONITOR** — Monitor for issues

CHANGE MANAGEMENT FOR INFRASTRUCTURE ENGINEERS

CONTROLLED CHANGES = STABLE SYSTEMS + HAPPY USERS

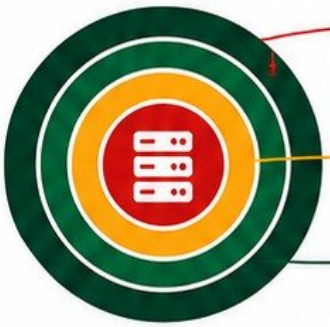
1 TYPES OF CHANGES

STANDARD CHANGES
Pre-approved, low-risk, routine and repeatable.
Examples: User creation, package updates.

NORMAL CHANGES
Require planning, review and approval.
Examples: Server upgrade, network changes.

EMERGENCY CHANGES
Urgent, unplanned changes to restore service.
Examples: Fix critical outage, security breach.

2 UNDERSTANDING BLAST RADIUS



HIGH IMPACT
Critical systems, many users, core business services.

MEDIUM IMPACT
Departmental systems, shared services.

LOW IMPACT
Non-critical systems, isolated components.

ALWAYS IDENTIFY WHAT CAN BE AFFECTED BEFORE YOU MAKE A CHANGE.

3 THE CHANGE MANAGEMENT PROCESS

1. CHANGE TICKET
- Document the reason, scope, systems affected and expected outcome.
- Get a unique change ID.

2. PEER REVIEW
- Another engineer reviews the plan.
- Catch mistakes before they reach production.

3. IMPLEMENTATION PLAN
- Define exact steps.
- List dependencies.
- Estimate time.
- Define success criteria.

4. MAINTENANCE WINDOW
- Schedule the change in a low-traffic period.
- Communicate time clearly.

5. BACKUP & ROLLBACK PLAN
- Take backups or snapshots.
- Document rollback steps.
- Test rollback if possible.

6. COMMUNICATION
- Notify stakeholders before the change.
- Share impact, time and expected downtime.
- Provide updates.

7. PERFORM THE CHANGE
- Follow the plan.
- Execute step-by-step.
- Monitor for issues.
- Stay calm and focused.

8. POST-CHANGE VERIFICATION
- Verify all systems and services.
- Check logs, metrics and alerts.
- Validate success criteria.

9. DOCUMENT THE RESULT
- Record what was changed and why.
- Note outcomes, issues and resolutions.

10. CLOSE THE CHANGE
- Update the ticket.
- Confirm with stakeholders.
- Close or move to post-implementation review.

KEY REMINDERS

- ✓ Plan every change. Don't change in production without a plan.
- ✓ Understand the blast radius. Small change, big impact is real.
- ✓ Back up and test rollback. Your safety net.
- ✓ Communicate early and clearly. No surprises.
- ✓ Verify and document everything. If it's not documented, it didn't happen.

COMMON MISTAKES

- ✗ Making changes without a ticket or plan.
- ✗ Ignoring dependencies and blast radius.
- ✗ No backup or rollback plan.
- ✗ Poor communication with stakeholders.
- ✗ Not verifying after the change.

JUNIOR ENGINEER TIP:
A GOOD CHANGE TODAY PREVENTS AN INCIDENT TOMORROW.

- ☑ PLAN. COMMUNICATE.
- ☑ EXECUTE. VERIFY.
- ☑ DOCUMENT. IMPROVE.

AUTOMATION AND INFRASTRUCTURE AS CODE IN DAILY OPERATIONS

AUTOMATE THE REPEATABLE. STANDARDIZE THE RELIABLE. PROTECT THE PRODUCTION.

1 WHY REPETITIVE MANUAL WORK CREATES RISK

- Humans make mistakes.
- Manual steps are inconsistent.
- Harder to audit and reproduce.
- Slower to scale and recover.
- Knowledge stays in people, not in systems.

AUTOMATION REDUCES RISK, IMPROVES SPEED AND DELIVERS CONSISTENCY.

2 KEY AUTOMATION TOOLS

SHELL SCRIPTS

- Great for simple and repetitive tasks.
- Easy to start with.
- Use version control.

ANSIBLE

- Agentless.
- Configuration management.
- Push-based automation.
- Great for servers.

TERRAFORM

- Infrastructure as Code (IaC).
- Declarative provisioning.
- Multi-cloud support.
- State aware.

CONFIGURATION MANAGEMENT

- Ensure desired state.
- Keep systems consistent.
- Drift detection and correction.

3 INFRASTRUCTURE AS CODE (IaC)

- Define infrastructure in code.
- Versioned, reviewable and repeatable.
- Enables consistency across environments.
- Makes infrastructure changes predictable.
- IaC is the foundation of modern operations.

4 GIT-BASED INFRASTRUCTURE CHANGES

- Store infrastructure code in Git.
- Track every change.
- Use branches for features and fixes.
- Pull requests for review.
- Merge after approval and testing.
- Single source of truth.

5 THE AUTOMATION WORKFLOW

1. **WRITE CODE** — Define the desired infrastructure or configuration.
2. **REVIEW CODE** — Peer review for correctness, security and best practices.
3. **PLAN / DRY RUN** — See what will change before anything is applied.
4. **APPROVE** — Get approval based on the plan and impact assessment.
5. **APPLY / EXECUTE** — Run the automation to make the changes safely.
6. **VERIFY** — Confirm the result matches the expected state.

6 REVIEW BEFORE EXECUTION

- Never run code you don't understand.
- Check for security issues, hardcoded secrets and misconfigurations.
- Validate logic, variables, permissions and impact.
- Use pull requests and checklists.

7 TERRAFORM PLANS

- terraform plan shows what will be created, changed or destroyed.
- Always review the plan output.
- Look for unexpected changes.
- No plan review = High risk.

8 IDEMPOTENCY

- Running automation multiple times should produce the same result.
- No unintended changes on repeated runs.
- Ensures stability and predictability.

AUTOMATING REPEATABLE OPERATIONS WITHOUT AUTOMATING MISTAKES

- Automate the boring, not the broken.
- Start small and test thoroughly.
- Keep human approval for high-impact changes.
- Monitor and log every automated action.
- Automation is a force multiplier, not a substitute for understanding.

JUNIOR ENGINEER TIP

AUTOMATE WITH INTENT. TEST LIKE YOU CARE. REVIEW LIKE A PARANOID. AUTOMATION AMPLIFIES BOTH GOOD PRACTICES AND BAD MISTAKES.

THE GOLDEN RULE

IF YOU WON'T TRUST IT IN PRODUCTION, DON'T AUTOMATE IT.

- ☑ Code it.
- ☑ Test it.
- ☑ Document it.
- ☑ Review it.
- ☑ Verify it.
- ☑ Improve it.

INCIDENT RESPONSE AND PRODUCTION TROUBLESHOOTING

THE INCIDENT RESPONSE LIFECYCLE

1. **DETECT**
   Alert fired, user report, monitoring or log signal.
2. **ASSESS IMPACT**
   Understand scope, severity, and who is affected.
3. **GATHER EVIDENCE**
   Collect logs, metrics, errors, screenshots and data.
4. **IDENTIFY RECENT CHANGES**
   Check deployments, configs, patches, or infrastructure changes.
5. **FORM A HYPOTHESIS**
   Make an educated guess about the root cause.
6. **RUN DIAGNOSTIC CHECKS**
   Test your hypothesis. Use commands, tools and queries.
7. **MITIGATE**
   Take immediate action to reduce impact or stop the bleeding.
8. **RECOVER**
   Restore the service to normal operation as quickly and safely as possible.
9. **VERIFY**
   Confirm the service is stable and all systems are healthy again.
10. **DOCUMENT**
    Record what happened, what you did, root cause and preventive actions.

WHEN TO ESCALATE

- Impact is high and you cannot mitigate quickly.
- You lack the access, tools, or expertise to resolve it.
- Time is critical and delay will increase damage.
- Multiple teams or domains are involved.
- You are unsure and need additional perspective.

ESCALATE EARLY.
ESCALATE CLEARLY.
ESCALATE WITH EVIDENCE.

USE THIS DIAGNOSTIC TOOLKIT

- `top / htop` — Check CPU and memory usage
- `df -h` — Check disk space
- `ss -tulnp` — Check listening ports
- `journalctl -xe` — Inspect system logs
- `kubectl get all -A` — Check Kubernetes resources
- `kubectl describe <resource>` — Get detailed resource events
- `curl -I http://service` — Test service health
- `dig example.com` — Test DNS resolution

WHY RESTORE FIRST?

During an active outage, the priority is to restore service and reduce user impact.

Root cause analysis can continue after the system is stable.

REMEMBER
It is better to have a working system with a known issue than a perfect RCA with unhappy users.

JUNIOR ENGINEER TIP

Stay calm. Follow the process. Communicate clearly.
Your goal is to bring stable service back to the users.

SECURITY OPERATIONS FOR INFRASTRUCTURE

SECURITY IS NOT A ONE-TIME TASK. IT IS A DAILY RESPONSIBILITY.

1 LEAST PRIVILEGE

- Give users and services only the access they need.
- Review permissions regularly.
- Remove unnecessary access.

2 SSH PROTECTION

- Use SSH keys, not passwords.
- Disable root login.
- Change default port.
- Use fail2ban or similar protection.

3 FIREWALL RULES

- Allow only required traffic.
- Deny everything else by default.
- Review rules periodically.

4 OPEN PORTS

- Check listening ports regularly.
- Close unused ports.
- Document why each port is open.

5 PATCH STATUS

- Keep OS, applications and dependencies up to date.
- Monitor for security advisories.
- Test before production patching.

6 SECRETS

- Never hardcode secrets.
- Use secret managers (Vault, AWS Secrets Manager, etc.).
- Restrict access to secrets.

7 SERVICE ACCOUNTS

- Use dedicated service accounts.
- Avoid using admin accounts for applications.
- Rotate credentials regularly.

8 PRIVILEGED ACCESS

- Limit sudo/root usage.
- Use just-in-time access when possible.
- Monitor and audit all privileged actions.

9 CERTIFICATE EXPIRATION

- Track SSL/TLS certificate expiry dates.
- Set alerts before expiration.
- Automate renewal where possible.

10 SUSPICIOUS AUTH ACTIVITY

- Monitor failed login attempts.
- Detect brute force and unusual patterns.
- Investigate immediately.
- Block malicious IPs.

11 HARDENING

- Disable unused services.
- Remove unnecessary packages.
- Enforce strong configurations.
- Follow security baselines.

12 LOGGING & AUDIT TRAILS

- Enable system and application logs.
- Centralize logs.
- Retain logs securely.
- Audit access and changes.

SECURITY IS PART OF DAILY INFRASTRUCTURE OPERATIONS

- ✔ Check security alerts and dashboards daily.
- ✔ Review access and configuration changes.
- ✔ Verify backups and encryption.
- ✔ Ensure compliance and audit requirements.
- ✔ Document and respond to issues.
- ✔ Promote a security-first culture.

JUNIOR ENGINEER TIP

If you do not protect the infrastructure, someone will find a way to break it.
THINK BEFORE YOU OPEN, CHANGE, OR SHARE ANYTHING.

TICKETS, DOCUMENTATION, HANDOVER AND OPERATIONAL DISCIPLINE

GOOD OPERATIONS ARE BUILT ON DISCIPLINE, CLARITY AND CONSISTENCY.

1	**WORKING FROM TICKETS**	• Every task should be tied to a ticket. • Understand the issue, impact, and requested outcome. • Update status, comments, and resolution.	★ **TIP** A good ticket tells the next engineer the full story.
2	**RECORDING COMMANDS AND CHANGES**	• Record the commands you run. • Note configuration changes and file edits. • Include timestamps and reasons.	★ **TIP** If it's not recorded, it didn't happen.
3	**RUNBOOKS**	• Step-by-step instructions for common tasks. • Include prerequisites, commands, and validation. • Keep runbooks simple, accurate and updated.	★ **TIP** Good runbooks prevent repeat mistakes.
4	**STANDARD OPERATING PROCEDURES (SOPS)**	• Define how work must be done. • Ensure consistency across the team. • Review and improve regularly.	★ **TIP** SOPs turn experience into repeatable quality.
5	**ARCHITECTURE DOCUMENTATION**	• Document systems, components, dependencies and data flows. • Keep diagrams and details up to date. • Store in a centralized, accessible place.	★ **TIP** Up-to-date docs save hours of guesswork.
6	**INCIDENT TIMELINES**	• Record key events with timestamps. • Track what was done and by whom. • Helps with root cause analysis and learning.	★ **TIP** Timelines tell the truth of what happened.
7	**ESCALATION NOTES**	• Note when and why an issue was escalated. • Include team or person escalated to. • Document guidance or actions received.	★ **TIP** Clear escalation notes avoid repeated escalations.
8	**SHIFT HANDOVERS**	• Share what was done and what's pending. • List active tickets, changes, and observations. • Communicate clearly, no assumptions.	★ **TIP** A good handover protects the next shift.
9	**OUTSTANDING RISKS**	• Highlight known issues and workarounds. • List potential impacts if unattended. • Keep risk register updated.	★ **TIP** Out of sight is not out of risk.
10	**COMMUNICATE CLEARLY WITH APPLICATION, NETWORK, SECURITY & DEVOPS TEAMS**	• Share facts, not blame. • Be concise, objective and timely. • Collaborate until the issue is resolved.	★ **TIP** Good communication solves problems faster.

WHY UNDOCUMENTED INFRASTRUCTURE BECOMES OPERATIONAL DEBT

- Harder troubleshooting
- Longer recovery times
- Repeated mistakes
- Higher risk
- Knowledge loss when people leave
- Slower onboarding
- Increased outages

DOCUMENTATION ISN'T EXTRA WORK — IT IS RISK REDUCTION.

JUNIOR ENGINEER TIP

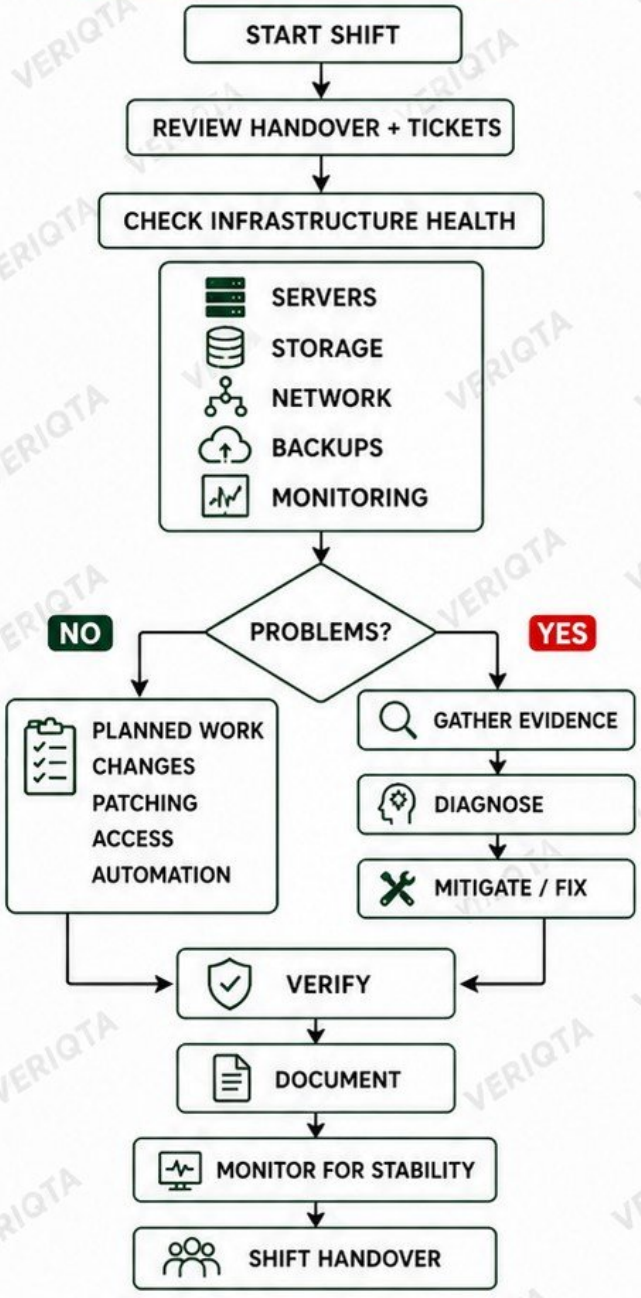
Write as if the next person needs to understand everything without asking you.
Your future self will thank you.

FINAL PRODUCTION SIMULATION

A DAY IN INFRASTRUCTURE OPERATIONS

1. START WITH THE MORNING HEALTH CHECK
2. DISCOVER A DISK AT 92%
3. INVESTIGATE ABNORMAL FILESYSTEM GROWTH
4. RECEIVE AN APPLICATION CONNECTIVITY TICKET
5. TEST DNS, ROUTING AND PORTS
6. FIND A FAILED BACKEND SERVICE
7. INSPECT LOGS
8. RESTORE THE SERVICE SAFELY
9. REVIEW A FAILED OVERNIGHT BACKUP
10. PROCESS A PLANNED INFRASTRUCTURE CHANGE
11. VERIFY MONITORING AFTER THE CHANGE
12. RESPOND TO A NEW CRITICAL ALERT
13. ESCALATE WHEN NECESSARY
14. DOCUMENT ACTIONS
15. UPDATE THE TICKET
16. COMPLETE END-OF-SHIFT HANDOVER

FINAL PAGE 20 MENTAL MODEL



KEY TAKEAWAY

A successful day is not about how many problems you solve, but how safely and consistently you keep systems available, secure and reliable.

JUNIOR ENGINEER TIP

Follow the process. Communicate early. Document everything. Your discipline today prevents outages tomorrow.